A)

Mode 1

In mode 1, I parallelized the outer loop *i* as each row of C is independent without any cross-iteration dependencies, with a balanced workload per iteration.

I used default(none) to enforce the explicit scoping and avoid accidental race conditions, stating that A, B, C and N are shared, with *i* being private.

This mode makes use of two different scheduling strategies that can be used interchangely. This makes use of either static chunk division, or dynamic chunks, where each chunk is decided by calculating $chunk = \frac{N}{4 * threads}$, since matrix multiplication has a uniform workload, static scheduling leads to improved performance as dynamic scheduling adds additional overhead.

Mode 2

This mode makes use of collapse(2), I parallelized the outer loop, this scales better with larger thread counts, for large N, performance was similar to Mode 1 because each row already provided sufficient work.

Mode 3A and 3B

When parallelizing the checksum calculation, you have the risk of encountering a race condition, therefore explicit synchronization was required.

This can be done using two ways, atomic (3A) and Critical (3B), while doing this, I also calculated the maxC and sumC.

3A

The checksum and sumC was updated using #pragma omp atomic update, while the maxC was updated using the #pragma omp compare, this is simpler for small things such as this.

3B

In 3B, the checksum, sumC and maxC are all handled in the critical block, serializing the entire inner loop body, with growing overhead depending on the thread count.

Mode 4

In mode 5, I implemented task-based parallelism, where the code is ran in this parallel block, that contains a single block, that parallelizes the inner loop j as tasks that are created per row.

After the multiplication it waits for the tasks to be finished with #pragma omp taskwait, then the additional tasks are computed.

Tasks are good for when the work units are irregular with dynamic work sizes, however this use case has uniform work, therefore tasks have higher overhead than work-sharing loops and slightly worse scalability.

Therefore tasks are not ideal for this use case.

Mode 5

This has a similar runtime, with a slight speedup due to the fact it only makes use of a single thread.

The outer loop was parallelized with #pragma omp for, with the inner loop having #pragma omp simd reduction(+:sum).

Mode 6

The code is the exact same as Mode 5, but we don’t set the thread number to 1, which makes use of parallelism for the threads and the data.

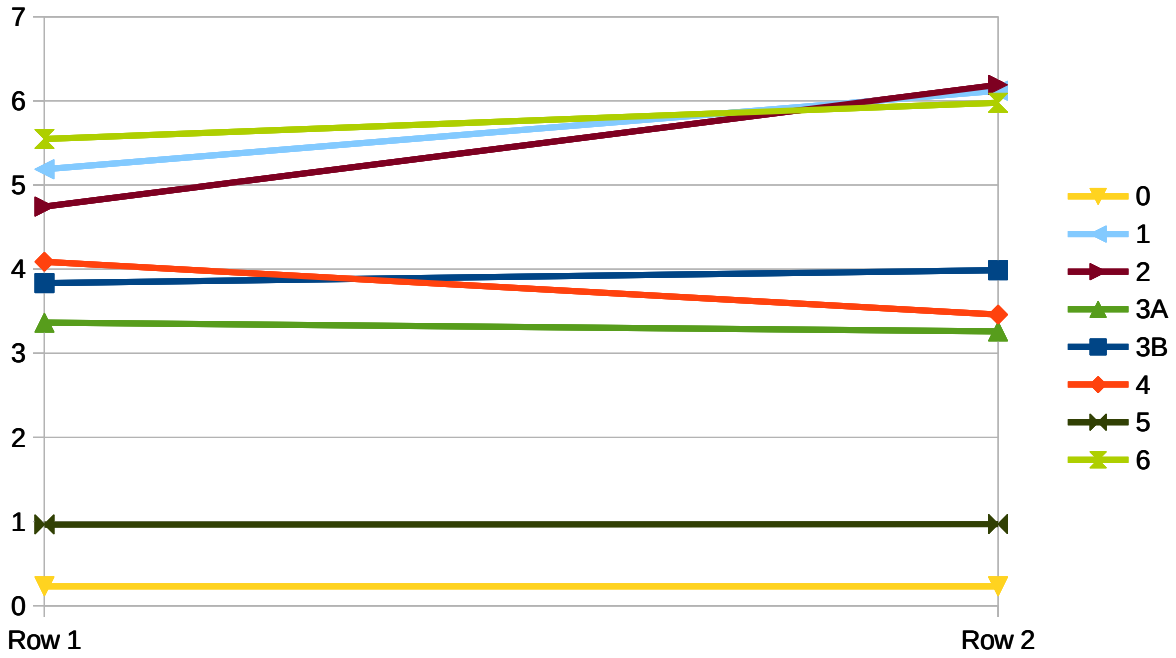
The combination of these two leads to the best overall performance.

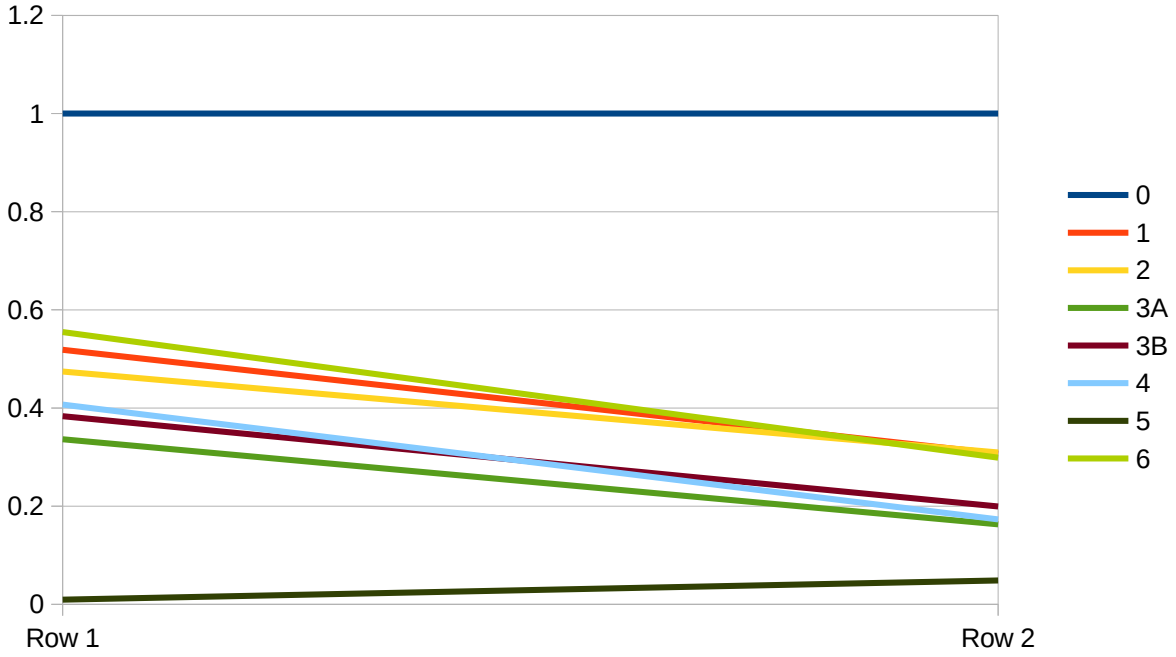
B)

N	Mode	Threads	Schedule	Times(s)	Speedup	Efficiency
512	0	1	NA	0.23	1	1
512	1	20	static	0.037580	6.12	0.3069
512	1	10	static	0.04434	5.1871	0.5187
512	2	20	static	0.037140	6.19	0.3096
512	2	10	static	0.04849	4.7432	0.4743
512	3A	20	static	0.070556	3.2598	0.1629
512	3A	10	static	0.06838	3.3635	0.3363
512	3B	20	static	0.057690	3.9868	0.1993
512	3B	10	static	0.06000	3.8333	0.3833
512	4	20	static	0.066469	3.4602	0.1730
512	4	10	static	0.05653	4.0686	0.4068
512	5	20	static	0.235776	0.97	0.0485

512	5	10	static	0.2381	0.9659	0.0096
512	6	20	static	0.038480	5.9771	0.2988
512	6	10	static	0.04147	5.5461	0.5546

C)





D)

1. Static Scheduling assigns loop iterations evenly to threads at the start, while dynamics scheduling allows the programmer to choose the chunks of iterations on the fly, static scheduling performs better in this case due to the fact that the workload is spread out uniformly.

2. Efficiency drops as the overhead of creating threads leads to additional costs.

3. For my use case, critical performed faster as using atomic variables leads to less serialization overhead than when using atomic.

4. Tasks are useful for when the ammount of work per iteration is irregular, which is not what happens in this case, causing more overhead.

5. The inner array calculations benefitted most, SIMD reduces the need for many threads as each thread handles multiple elements at once handles multiple elements at once.